# TryParse: Study Notes

PROG1337 · Oct 20, 2026 · @SC

## Why we need TryParse

`Convert.ToInt32` crashes the whole program if the user types anything that isn't a whole number. TryParse checks first and never crashes.

```
Console.Write("Age: ");
int age = Convert.ToInt32(Console.ReadLine());
```

If the user types `abc`, the program stops with this runtime error:

> FormatException: The input string 'abc' was not in a correct format.

The same crash happens for a decimal like `12.5`, and when the user just presses Enter (the message shows `''`, an empty string). Users make typos all the time, so a program that crashes on a typo isn't finished.

## How int.TryParse works

`int.TryParse` tries to turn text into a whole number. It gives back **two** things: a `bool` that says whether it worked, and the number itself.

```
Console.Write("Age: ");
string input = Console.ReadLine();
bool isNumber = int.TryParse(input, out int age);
```

Read the last line from the inside out:

1. `input` is the text to check.
2. `out int age` declares a new variable, `age`. TryParse puts the number into it. Think of it as handing TryParse an empty box to fill.
3. TryParse returns `true` if it worked and `false` if it didn't. That answer goes into `isNumber`.

| The user types | isNumber | age |
|---|---|---|
| 25 | True | 25 |
| abc | False | 0 |
| 12.5 | False | 0 |
| (just Enter) | False | 0 |

**When it fails, the number is 0.** That 0 doesn't mean the user typed 0. Only trust the number when the bool is `true`.

## Using the answer with if

Because `isNumber` is a `bool`, it can go straight into an `if`.

```
if (isNumber)
{
    Console.WriteLine($"Next year you'll be {age + 1}.");
}
else
{
    Console.WriteLine("That isn't a whole number.");
}
```

This no longer crashes, but it only checks once. If the answer is bad, the program just moves on. To ask again until the answer is good, we need a loop.

## The standard pattern: keep asking until it's valid

From now on, this is how we read a number from the user. Ask once, then keep asking **while** it isn't a number, or it's too small, or it's too big.

```
Console.Write("Age (0 to 120): ");
string input = Console.ReadLine();
bool isNumber = int.TryParse(input, out int age);

while (!isNumber || age < 0 || age > 120)
{
    Console.WriteLine("That isn't a whole number from 0 to 120. Try again.");
    Console.Write("Age (0 to 120): ");
    input = Console.ReadLine();
    isNumber = int.TryParse(input, out age);
}
```

Why each part is there:

- **`while`, not `if`:** an `if` checks once, so a second bad answer gets through. A `while` keeps checking until the answer is good.
- **`!isNumber`:** `!` means "not", so this part is true when it is NOT a number.
- **`||` (or):** any one problem is enough to ask again. With `&&`, every problem would have to be true at once, which never happens, so bad answers would get through.
- **`isNumber =` inside the loop:** the loop's condition must be updated with the new answer. Without it, `isNumber` stays false forever and the loop never ends.
- **Why the order is safe:** if the text isn't a number, `age` is 0, but `!isNumber` is already true, so the loop asks again either way.

## out int age vs out age

`out int age` **declares** a new variable. You can only declare a name once, so every later TryParse into the same variable says `out age`, with no `int`.

```
bool isNumber = int.TryParse(input, out int age);   // first time: declare
isNumber = int.TryParse(input, out age);            // after that: reuse
```

If you write `out int age` a second time, you get a compile-time error:

> CS0136: A local or parameter named 'age' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter

The fix is to remove the `int`.

## double.TryParse for decimals

`double.TryParse` works exactly the same way. Only the type changes.

```
Console.Write("Price: ");
string input = Console.ReadLine();
bool isPrice = double.TryParse(input, out double price);

while (!isPrice || price < 0)
{
    Console.WriteLine("That isn't a valid price. Try again.");
    Console.Write("Price: ");
    input = Console.ReadLine();
    isPrice = double.TryParse(input, out price);
}

Console.WriteLine($"With 15% HST: {price * 1.15}");
```

Typing `20` prints `With 15% HST: 23`.

**Regional settings matter.** On a computer set to French (Canada), `double.TryParse` expects a decimal comma: `12,5`, not `12.5`. If `12.5` keeps getting rejected on your computer, try `12,5`.

## Common mistakes

| What you see | Why | Fix |
|---|---|---|
| A crash: "The input string 'abc' was not in a correct format" | Still using `Convert.ToInt32` on what the user typed | Use `int.TryParse` |
| CS0136: "A local or parameter named 'age' cannot be declared in this scope..." | `out int age` written a second time | Use `out age`, with no `int` |
| After one bad answer, the loop never ends | The TryParse inside the loop doesn't store its answer | `isNumber = int.TryParse(…)` |
| Even good numbers keep asking | The `!` is missing before `isNumber` | `while (!isNumber \|\| …)` |
| `abc` is accepted, and the number is 0 | `&&` instead of `\|\|` in the condition | Use `\|\|`: any one problem means ask again |
| A second bad answer gets through | `if` instead of `while` | Use `while` |

## Check yourself

Work these out on paper first, then type them into Visual Studio to check. Answers are below.

1. What do `isNumber` and `n` hold after `bool isNumber = int.TryParse("12.5", out int n);`?
2. What about `int.TryParse(" 7 ", out int n)`, with spaces around the 7?

3. What about `int.TryParse("-3", out int n)`?
4. What about `double.TryParse("12.5", out double n)` on a computer set to English (Canada)?
5. Write the `while` condition that keeps asking until the user types a whole number from 1 to 6. Use `isNumber` and `roll`.
6. This line is inside a keep-asking loop. What's wrong with it? `int.TryParse(input, out roll);`
7. This line is inside the same loop. What error do you get, and why? `isNumber = int.TryParse(input, out int roll);`

## Answers

1. `False` and `0`. 12.5 isn't a whole number, so `int.TryParse` fails and `n` is 0.
2. `True` and `7`. TryParse ignores spaces around the number.
3. `True` and `-3`. Negative numbers are whole numbers. If you don't want them, your range check has to reject them.
4. `True` and `12.5`. On French (Canada) settings it would be `False`, and you'd type `12,5`.
5. `while (!isNumber || roll < 1 || roll > 6)`
6. The answer isn't stored in `isNumber`, so `isNumber` never changes and the loop never ends. It should be `isNumber = int.TryParse(input, out roll);`
7. CS0136. `roll` was already declared by the first TryParse before the loop, so inside the loop it must be `out roll`, with no `int`.